Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 02.12.2023.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_Test1_GrupaI_IT1g2021**
8. **Minimalni broj bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka je 5**.
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Pronalaženje elementa u redu (Queue Peek)*

Napisati metodu koja implementira standardan način za pronalaženje elementa sa vrha reda.

Dodatne napomene za rešavanje zadatka:

- Algoritam napisati u okviru zakomentarisane metode ***peek(int value).***
- Pomenuta metoda nalazi se u okviru klase ***Queue*** koja se nalazi u pridruženom template-u.
- Obezbediti kod u Main metodi za testiranje algoritma.

Zadatak 2 – *Pomeranje 0 na kraj dvostruko spregnute liste*

Kreirati funkciju koja će proći kroz dvostruko spregnutu listu i pomeriti sve 0 na kraj. Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Dodatni zahtevi za rešavanje zadatka:

- Algoritam mora da rezultuje u promeni vrednosti postojeće dvostruko spregnute liste.
- Mogu se menjati samo vrednosti, a mogu se prevezivati i kompletni čvorovi.
- Zahtevan potpis metode: ***public void MoveZerosToEnd()***

Primer:

- Input: $9 \leftrightarrow -4 \leftrightarrow 0 \leftrightarrow 1 \leftrightarrow 0 \leftrightarrow 0 \leftrightarrow 7 \leftrightarrow 0$
- Output: $9 \leftrightarrow -4 \leftrightarrow 1 \leftrightarrow 7 \leftrightarrow 0 \leftrightarrow 0 \leftrightarrow 0 \leftrightarrow 0]$
- Input: $1 \leftrightarrow 0 \leftrightarrow 0 \leftrightarrow 1 \leftrightarrow 0 \leftrightarrow 0$
- Output:$1 \leftrightarrow 1 \leftrightarrow 0 \leftrightarrow 0 \leftrightarrow 0 \leftrightarrow 0$

Zadatak 3 – *Excessive broj*

Kreiranje funkcije koja proverava da li je neki broj ***excessive number***. Excessive broj je broj kod kojeg suma svih njegovih delioca ne sme biti veća od samog broja. Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Dodatni zahtevi za rešavanje zadatka:

- Algoritam treba da vrati bool vrednost.
- Cifre prosleđenog broja neophodno je smestiti u niz. Ukoliko se koristi dinamički niz, koristiti funkciju ToArray() kako bi se od njega dobio statički niz.
- Zahtevan potpis metode: ***public bool IsNumberExcessive(long number)***

Primer:

- Input 1: 12
- Objašnjenje: 1 + 2 + 3 + 4 + 6 = 16
- Output 1: false

- Input 2: 15
- Objašnjenje: 1 + 3 + 5 = 9
- Output 2: true